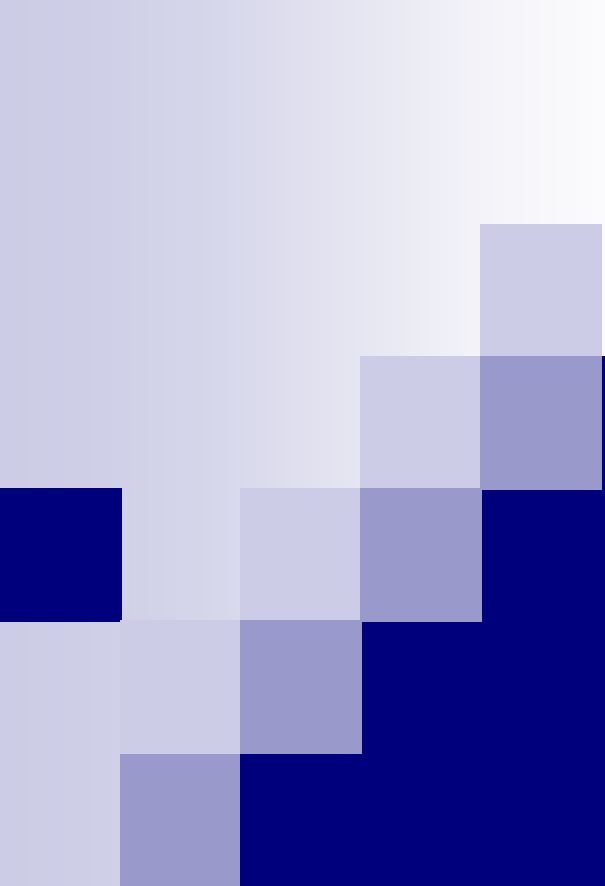




Laboratorio di BD1

Il linguaggio SQL

(Selezione)

Docente: Daniele Riboni
Tutor: Fabio Pili e Luca Garau

Dipartimento di Matematica e Informatica
Università degli Studi di Cagliari

Nella precedente lezione:

1. Vincoli inter-relazionali (*Foreign Key – References*)
2. Politiche di reazione (*On Update/Delete*)
 1. Cascade
 2. Set *NULL*
 3. Set *Default*
 4. *Restrict*
 5. *No Action*
3. Modifica degli schemi (*Drop/Alter*)

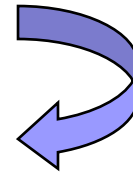


Argomenti delle prossime lezioni

1. Proprietà del linguaggio SQL, selezione e ordinamento
2. Join tra due tabelle
3. Join multipli
4. Query nidificate e subquery correlate
5. Funzioni aggregate
6. Raggruppamento
7. Operatori insiemistici
8. Viste

Proprietà del linguaggio SQL

- SQL esprime le interrogazioni in modo *dichiarativo*, specificando cioè le proprietà del risultato e non il modo in cui ottenerlo.



In ciò SQL segue i principi del calcolo relazionale e si contrappone ai linguaggi di interrogazione procedurali, come l'algebra relazionale.

Proprietà del linguaggio SQL

- SQL fornisce quindi

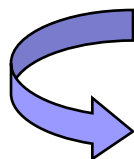
un'interfaccia dichiarativa di alto livello,

lasciando al DBMS il compito di individuare
la migliore strategia di accesso ai dati ovvero
la sequenza di operazioni da eseguire
per rispondere all'interrogazione

(query optimizer)

Proprietà del linguaggio SQL

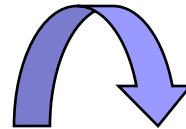
- **N.B.** Il linguaggio SQL
è “*relazionalmente completo*”



permette di esprimere
tutte le interrogazioni formulabili
in Algebra Relazionale

Proprietà del linguaggio SQL

- SQL opera su *multiset* (o *bag*) di tuple, cioè su insiemi di tuple che possono contenere duplicati



in questo, le tabelle SQL (ottenute come risultato di una query) si differenziano dalle relazioni del modello relazionale

Proprietà del linguaggio SQL: Recap

- Linguaggio dichiarativo
- Presenza del Query Optimizer
- Linguaggio relazionalmente completo
- Opera su multiset/multibag di tuple

Istruzione SELECT

- Sintassi di base:

SELECT *ListaAttributi*

FROM *ListaTabelle*

[WHERE *Condizione*]

Istruzione SELECT

***SELECT ListaAttributi FROM ListaTabelle
[WHERE Condizione]***

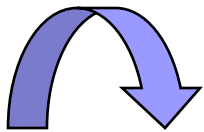
ListaTabelle ☐ una o più tabelle/viste ($T_1, \dots, T_n$)

ListaAttributi ☐ uno o più attributi delle tabelle elencate
(ovvero espressioni definite su tali attributi)

Condizione ☐ combinazione booleana di predicati
semplici, predicati di join e sub-query

Istruzione SELECT

SELECT *ListaAttributi* FROM $T_1, \dots, T_n$
WHERE *Condizione*

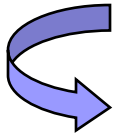


Significato dell'interrogazione:

$\pi_{\text{ListaAttributi}} (\sigma_{\text{Condizione}} (T_1 \times \dots \times T_n))$

Istruzione *SELECT*

$\pi_{\text{ListaAttributi}} (\sigma_{\text{Condizione}} (T_1 \times \dots \times T_n))$



Dal prodotto cartesiano delle tabelle elencate nella *clausola FROM*

vengono selezionate le tuple che soddisfano la condizione espressa nella *clausola WHERE*

e, per ciascuna tupla selezionata, si considerano solo gli attributi specificati in *ListaAttributi*

*Istruzione **SELECT**: esempi*

- *Schema di riferimento:*

Studenti (Matricola, Cognome, Nome, CittàNascita,
CittàResidenza, Anno)

Corsi (Codice, Nome, Docente)

Esami (Studente, Corso, Data, Voto)

FOREIGN KEY: Studente REFERENCES Studenti

FOREIGN KEY: Corso REFERENCES Corsi

Istruzione SELECT: esempi

- *Matricola e cognome
degli studenti del secondo anno:*

SELECT Matricola, Cognome

FROM Studenti

WHERE Anno = 2

Istruzione SELECT: esempi

- *Codice dei corsi
tenuti dal docente 'D4':*

SELECT Codice

FROM Corsi

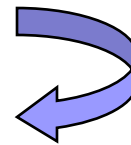
WHERE Docente = 'D4'

Istruzione SELECT: esempi

- *Codice di tutti i corsi:*

```
SELECT Codice FROM Corsi
```

In assenza di una condizione
di selezione (clausola WHERE)



tutte le tuple della tabella specificata
nella clausola FROM contribuiscono al risultato

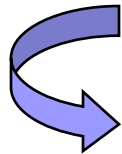
Istruzione SELECT: esempi

- *Esami con voto compreso tra 24 e 28:*

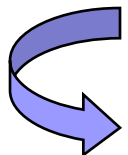
SELECT *  *tutti gli attributi*
FROM Esami
WHERE Voto >= 24 AND Voto <= 28

Predicati semplici

- La clausola WHERE ammette come argomento una combinazione booleana di *predicati semplici*



ciascun predicato può contenere
uno dei seguenti operatori



Predicati semplici

■ OPERATORI RELAZIONALI:

espressione OP-REL costante oppure
espressione1 OP-REL espressione2

dove

- l'espressione è costruita a partire dai valori degli attributi (nel caso più semplice è rappresentata dal nome di un attributo)
- $OP-REL \in \{ =, <>, >, >=, <, <= \}$

Predicati semplici

- Esempi:

```
SELECT * FROM Studenti  
        WHERE CittàResidenza = 'Cagliari'
```

```
SELECT * FROM Studenti  
        WHERE CittàResidenza = CittàNascita
```

Predicati semplici

- OPERATORE DI RANGE:

*WHERE attributo [NOT] BETWEEN
costante1 AND costante2*

Es.: Esami con voto compreso tra 24 e 28:

```
SELECT * FROM Esami  
WHERE Voto BETWEEN 24 AND 28
```

Predicati semplici

- OPERATORE DI SET:

attributo [NOT] IN
(costante1,, costanteN)

Es.: Esami con voto pari a 29, 30 o 33 (lode):

```
SELECT * FROM Esami  
WHERE Voto IN (29, 30, 33)
```

Predicati semplici

- **OPERATORI QUANTIFICATI: *ANY* e *ALL***

attributo OP-REL < ANY | ALL >
(costante1,, costanteN)

Es.: Esami con voto pari a 29, 30 o 33:

```
SELECT * FROM Esami  
WHERE Voto = ANY (29, 30, 33)
```

Predicati semplici

- OPERATORI QUANTIFICATI: *ANY* e *ALL*

attributo OP-REL < ANY | ALL >
(*costante1*,, *costanteN*)

La sintassi di PostgreSQL richiede *array expression*:

```
SELECT * FROM segreteria.esami  
WHERE voto = ANY('{29,30,33}');
```


Predicati semplici

- OPERATORI QUANTIFICATI: *ANY* e *ALL*

attributo OP-REL < ANY | ALL >
(costante1,, costanteN)

Es.: Esami con voto diverso da 29, 30 o 33:

```
SELECT * FROM esami  
WHERE Voto <> ALL (29, 30, 33)
```

Predicati semplici

- **OPERATORE DI CONFRONTO FRA STRINGHE:**

attributo [NOT] LIKE stringa

dove *stringa* può contenere
i caratteri speciali:

- (carattere arbitrario)
- % (stringa arbitraria, anche vuota)

Predicati semplici

- Esempio:

Studenti il cui nome
inizia con 'A' e termina con 'o':

```
SELECT * FROM Studenti  
WHERE Nome LIKE 'A%o'
```

Predicati semplici

- Esempio:

Studenti che hanno 'r'
come terza lettera del cognome:

```
SELECT * FROM Studenti  
WHERE Cognome LIKE '___r%'
```

Predicati semplici

- OPERATORE DI CONFRONTO CON NULL:

attributo IS [NOT] NULL

Es.: Studenti con l'attributo *CittàResidenza*
non specificato:

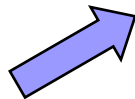
```
SELECT * FROM Studenti  
WHERE CittàResidenza IS NULL
```

Predicati semplici

- Due o più predicati possono essere combinati con gli operatori logici AND e OR.

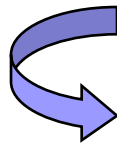
Es.: Studenti del terzo anno nati a Cagliari:

```
SELECT * FROM Studenti  
        WHERE Anno = 3  
        AND CittàNascita = 'Cagliari'
```



Predicati semplici

- **N.B.** La sintassi non definisce una relazione di precedenza tra gli operatori AND e OR.



Se l'interrogazione richiede l'uso di entrambi gli operatori, conviene quindi esplicitare l'ordine di valutazione mediante parentesi.

Operatori - Recap

In sintesi, la clausola *WHERE*:

1. Ci consente di filtrare le tuple nel risultato delle query
2. Supporta l'uso di vari operatori
 1. Operatore di *Range* □ attr *[NOT] BETWEEN c1 AND c2*
 2. Operatore di *Set* □ attr *[NOT] IN (c1, c2, ..., cn)*
 3. Operatori *Quantificati* □ attr OP *ANY / ALL* ('{ c1, c2, ..., cn }')
 4. Operatore di *confronto tra stringhe* □ attr *[NOT] LIKE 'string'*
 5. Operatore di *confronto con NULL* □ attr *IS [NOT] NULL*

Eliminazione dei duplicati

- SQL opera su *multiset* (o *bag*) di tuple
 - in una tabella (su cui non è stata definita una chiave), e nel risultato di un'interrogazione *possono comparire due o più tuple con identici valori degli attributi*

Eliminazione dei duplicati

- Eventuali duplicati possono essere eliminati dal risultato di un'interrogazione:

SELECT [DISTINCT | ALL] FROM

DISTINCT ☐ i duplicati vengono eliminati

ALL ☐ i duplicati vengono mantenuti
(opzione di default)

Eliminazione dei duplicati

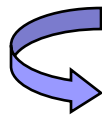
- SQL non elimina automaticamente i duplicati principalmente per ragioni di efficienza (l'eliminazione dei duplicati è un'operazione costosa)
- Inoltre, in alcuni casi l'utente può voler mantenere i duplicati nel risultato dell'interrogazione (specie quando si usano funzioni di aggregazione)

Ordinamento del risultato

- SQL permette di specificare un ordinamento delle righe del risultato di un'interrogazione:

SELECT FROM WHERE

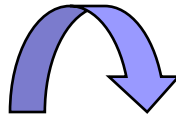
ORDER BY *ListaOrder*



uno o più attributi (specificati tramite il nome o la posizione nell'argomento della SELECT)

Ordinamento del risultato

- Le righe vengono ordinate in base al primo attributo di *ListaOrder*



a parità di valore di questo attributo,
si considerano i valori degli attributi
successivi, in sequenza

Ordinamento del risultato

- Per ciascun attributo di ordinamento, si possono specificare le opzioni:

DESC ☐ ordinamento decrescente

ASC ☐ ordinamento crescente
(*opzione di default*)

Ordinamento del risultato: esempio

- *Studenti residenti a Cagliari,
ordinati in senso crescente
rispetto all'anno di corso:*

```
SELECT * FROM Studenti  
WHERE CittàResidenza = 'Cagliari'  
ORDER BY Anno
```

Ordinamento del risultato: esempio

- *Esami del corso 'C1' ordinati in senso decrescente rispetto al voto espresso in centesimi e, a parità di voto, in senso crescente rispetto alla matricola:*

```
SELECT Studente, Data, (100*Voto)/30  
FROM Esami WHERE Corso = 'C1'  
ORDER BY 3 DESC, Studente
```

*terzo argomento
della SELECT*



Ridenominazione

- È possibile assegnare un *Alias* agli attributi/espressioni specificati come argomento della SELECT

**SELECT Attributo₁ AS A₁,,
Attributo_k AS A_k FROM**

Ridenominazione

- Esempi:

```
SELECT Cognome, Nome,  
       CittàResidenza AS Città  
FROM Studenti WHERE Anno = 1
```

```
SELECT Cognome, (Stipendio + Premio) AS Entrate  
FROM Impiegati  
ORDER BY Entrate
```

Ridenominazione

- È inoltre possibile assegnare un *Alias* alle tabelle specificate nella clausola FROM:

SELECT

**FROM Tabella₁ [AS] T₁,,
Tabella_n [AS] T_n**

Ridenominazione

- Esempio:

```
SELECT S.Cognome, S.Nome,  
FROM Studenti S  
WHERE S.Anno = 1
```

- Come vedremo, la ridenominazione è indispensabile nelle interrogazioni in cui occorre considerare due volte una stessa relazione.

Operatore punto

- Si usa per indicare la tabella da cui viene estratto un attributo:

SELECT **Impiegati**.Cognome

- Permette di distinguere due o più attributi con lo stesso nome contenuti in tabelle diverse:

SELECT **T1**.Nome, **T2**.Nome

Esercizi

- Esercitazione 2 [Schema 'segreteria']
 - Esercizio 1